

Word count: 5354 / 5k

Building a Quantum Simulator in Rust

Daniel J. Maynard



Swansea University
Prifysgol Abertawe

Contents

Abstract	3
1 Introduction	4
2 Background	6
2.1 Qubits	6
2.2 Gates	6
2.3 Statevectors	7
2.4 Multi-qubit gates	7
2.5 Kronecker products	8
2.6 Direct pair multiplication (DPM)	9
2.7 Gottesman-Knill theorem	10
3 Design & Implementation	11
3.1 Development environment	11
3.2 Library Architecture & Implementation	11
3.2.1 Circuit Representation	11
3.2.2 Statevector	12
3.2.3 Kernels	13
3.2.3.1 Portable DPM kernels	13
3.2.3.2 FMA DPM kernels	14
3.2.3.3 AVX DPM kernels	14
3.3 Testing, benchmarking and observability	15
4 Results	17
4.1 Experimental configuration & Methodology	17
4.2 Full-system matrix against direct indexing	17
4.2.1 Hadamard gate execution time	18
4.2.2 CNOT gate execution time	18
4.2.3 Hadamard and CNOT gate peak memory usage	19
4.3 Hardware accelerated kernels	19
4.3.1 Hadamard gate execution time over number of qubits	19
4.3.2 Hadamard gate execution time over target qubit	20
5 Analysis and discussion	22
5.1 Full-system matrix expansion	22
5.2 Hardware acceleration	23
6 Conclusion	25
Bibliography	26
Appendix	26

Abstract

Note: Final addition, 5-6 concise sentences.

This paper investigates different approaches to implementing quantum circuit statevector simulators. This was done by developing a simulator from scratch and evaluating it through benchmarking and testing. The implementation operates directly on pairs of amplitudes rather than constructing full-system matrices. It also utilises fused multiply-add and SIMD instructions. **Results sentence. Discussion sentence.**

1 Introduction

Quantum computing is an emerging technology that leverages principles of quantum mechanics at the hardware level to create computers that are capable of performing computations in a fundamentally different way to classical machines. For specific classes of problems this ‘quantum advantage’ can be utilised by carefully designed algorithms to achieve non-trivial improvements in computational complexity [1].

However, the field is in the Noisy Intermediate Scale Quantum (NISQ) era, characterized by processors that are constrained in size (number of qubits) and lack fault tolerance. In this context, a lack of fault tolerance means that processors are not yet stable enough to correct the errors introduced by cumulative external noise [2]. Beyond these technical limitations, the hardware remains a scarce resource; it is extremely expensive to build, maintain and run a quantum computer. While cloud-based providers, such as Microsoft Azure Quantum and Amazon Braket, have improved access to quantum computing by offering Quantum-as-a-Service (QaaS), access to high-performance chips can involve long wait times and significant usage fees, which can pose an obstacle to development [3]. Due to these constraints, the ability to simulate quantum circuits on classical hardware currently remains the most practical way to develop and test quantum algorithms. Quantum circuit simulation involves evaluating the effect of a series of logic gates mathematically on a representation of a quantum state. This approach provides an environment free of the typical environmental noise inherent to current quantum hardware and is far more cost effective [4].

However, simulation comes with its own set of limitations. The core issue is the memory cost of describing a quantum state classically. The base units of quantum information are qubits (quantum bits) and the direct conventional approach is known as statevector simulation in which to fully describe a quantum system of n qubits requires tracking 2^n individual complex amplitudes. Even on a supercomputer such as Summit [5], we can only conduct algorithmic simulations of a quantum circuit up to 47 qubits, which requires 2.8 petabytes of memory [4]. Mitigating this scaling is an active area of research with many approaches that trade exactness for tractability, but in the end this limitation is not escapable, as it is an inherent consequence of mapping quantum information onto classical hardware.

To escape this exponential scaling, alternative techniques sacrifice flexibility or exactness to make high-qubit simulation feasible. For example the stabiliser formalism allows certain classes of quantum circuit to be simulated efficiently, but cannot handle arbitrary algorithms. Because of this, statevector simulation remains the primary approach when flexibility is required making its computational cost a key bottleneck.

To explore how this bottleneck can be mitigated, this paper aims to investigate the implementation of quantum circuit simulators, with a primary focus on statevector

simulation techniques alongside exploring stabiliser simulation. For this purpose a simulator library is developed from scratch with support for single-qubit gates, controlled two-qubit gates and measurement. Different approaches for applying quantum operations to a quantum state are investigated and evaluated. The paper also explores the implementation and effects of hardware acceleration on execution time.

The specific objectives are:

- Implement a functional quantum circuit simulator using a statevector representation.
- Implement common single-qubit gates, controlled two-qubit gates and measurement.
- Implement multiple high-level approaches to computing the application of quantum operations including full-system matrix and direct pair multiplication.
- Implement multiple low-level approaches to computing the application of quantum operations utilising hardware acceleration.
- Implement a secondary stabiliser simulator.
- Evaluate the computational characteristics of approaches.

Together, these objectives allow the examination of the computational cost of statevector simulation at both the algorithmic and implementation level as well an evaluation against stabiliser techniques. The following chapters will go on to discuss the underlying theory, design and implementation of the simulator, the methods used to evaluate its performance and the results of these experiments.

2 Background

Note: Section 2.5 supported Tennessee University reference.

This section will introduce the basics of quantum computing using statevectors and some core principles behind some of the techniques used before introducing an alternative approach to simulation. The material presented in the first four subsections is a brief introduction to the quantum computing foundations required by this project and follows Nielsen and Chuang [6].

2.1 Qubits

A qubit can be considered the quantum analogue of a bit. Unlike a classical bit which has a value of either one or zero, a qubit is capable of existing as a linear combination of both values. More precisely a linear combination of two basis states, $|0\rangle$ and $|1\rangle$. However, when measured this qubit will collapse into one of the two basis states probabilistically. The definition of a qubit is show below:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Where α and β are complex numbers whose square moduli give the probability of each outcome during collapse. For a valid qubit the total probability, or the probability of there being an outcome at all, must be one. Which gives rise to the normalisation condition:

$$|\alpha|^2 + |\beta|^2 = 1$$

2.2 Gates

Quantum gates or operators as they are also known, are operations that are applied to qubits and form the building blocks of quantum circuits. A gate is represented by a matrix and applied using matrix-vector multiplication. This can be visualised in detail by using vector notation:

$$|\psi'\rangle = U|\psi\rangle = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} a\alpha + b\beta \\ c\alpha + d\beta \end{bmatrix}$$

where $|\psi'\rangle$ is the resulting changed qubit, U is an arbitrary gate matrix, a, b, c and d are the matrix coefficients and α and β are still the probability amplitudes.

All gate matrices (aside from measurement gates) are unitary, meaning they preserve the normalisation of the state and can be reversed by simply applying them again.

Common single-qubit gates include the Pauli X , Y and Z gates as well as the Hadamard gate H :

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

2.3 Statevectors

So far a single qubit has been used however, there is not much computation that can be done with only a single qubit. Quantum algorithms typically run on many-qubit systems, so while $|\psi\rangle$ represented a qubit it was actually a single qubit statevector. A statevector represents a quantum state of n -qubits by using 2^n complex amplitudes, and when generalising across n qubits it is conventionally written in the form:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle$$

where $|i\rangle$ is the i th basis state and α_i is its associated probability amplitude. Basis states are typically numbered in binary notation rather than decimal i.e $|010\rangle$. The normalisation condition can then be generalised to:

$$\sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1$$

To represent a statevector computationally, a fixed ordering of the basis states is assigned, either big or little endian. This allows the state to be represented in a more computation friendly format as a column vector:

$$|\psi\rangle = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{2^n-1} \end{bmatrix} \in \mathbb{C}^{2^n}$$

As a consequence the space complexity for a statevector must be $O(2^n)$.

2.4 Multi-qubit gates

Just as with single-qubit gates a gate that acts on multiple qubits will be a matrix. The number of qubits it operates on has the following relationship with its dimensions, a k qubit gate will be represented by a $2^k \times 2^k$ matrix. A common two-qubit gate is the SWAP gate:

$$\text{SWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

Which has the property of swapping the amplitudes of two qubits unconditionally.

Another key type of gate is the controlled gate, this is what allows branching and more complex behaviour in quantum circuits. Common controlled two-qubit gates include the controlled Pauli matrices:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad \text{CY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -i \\ 0 & 0 & i & 0 \end{bmatrix}, \quad \text{CZ} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix},$$

The way these gates function is that they ‘take’ two qubits, a control and a target qubit. If the control qubit is one then they apply the single-qubit pauli gate to the target qubit. However this perhaps overstates the simplicity when one takes into account the fact that in a quantum circuit the control qubit can be in a superposition of both one and zero simultaneously.

2.5 Kronecker products

In section 2.2 it was stated that to apply a gate to a qubit matrix-vector multiplication is used. **Fix** But this isn’t as simple once the state is made up of multiple qubits, since valid matrix-vector multiplication requires that the number of columns in the matrix are equal to the number of rows in the column vector. So to apply a gate matrix to a n -qubit statevector, the matrix must be expanded to a full-system matrix with 2^n columns. This is done by using the kronecker product.

The Kronecker product, denoted by $\otimes$, combines two smaller matrices to produce a larger matrix, where every element in the first matrix is multiplied by the second matrix. If matrix A has dimensions $a \times b$ and matrix B has dimensions $c \times d$ then $A \otimes B$ produces a matrix with dimensions $ac \times bd$.

Therefore it can be used to expand a gate matrix to a full-system matrix that can be applied to a quantum systems statevector. For a single-qubit gate this can be described as:

$$\tilde{U}_t = I^{\otimes t} \otimes U \otimes I^{(n-t-1)}$$

Where $\tilde{U}_t$ is the full-system matrix of a single-qubit gate with a target qubit index t and $I^{\otimes t}$ denotes the Kronecker product of t identity matrices. An identity matrix produces no effect on the statevector which is why the gate can be expanded this way without distorting the operation. For controlled two-qubit gates there exists a more complex formula:

$$\widetilde{\text{CU}}_{c,t} = P_0 \otimes I + P_1 \otimes U$$

Where $\widetilde{\text{CU}}_{c,t}$ is the full-system matrix of a controlled two-qubit gate, and the projectors P_0 and P_1 are defined as:

$$P_0 = |0\rangle\langle 0| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad P_1 = |1\rangle\langle 1| = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

Since $I|\psi\rangle = |\psi\rangle$ leaving the state unchanged, $P_0 \otimes I$ represents the branch where the controlled gate is not triggered, P_0 projects the identity matrix onto the corresponding amplitudes. By the same token, P_1 maps the operation U onto the amplitudes where the control qubit is activated. Notably this requires two full-system matrices to be constructed and then combined before being applied.

It is evident that gate application on many-qubit statevectors has an enormous computational cost. With the full-system matrices having a space complexity of $O(4^n)$ and controlled two-qubit gates requiring two full-system matrices. In addition while these operations are being executed the statevector still needs to remain in memory with its own space complexity of $O(2^n)$.

2.6 Direct pair multiplication (DPM)

Rather than attempting to apply a full-system matrix to the entire statevector, there is an approach that limits a simulation's space complexity to that of the statevector's while also improving execution time with workloads much more suited to modern processors.

This approach decomposes the 2^n -element statevector into 2^{n-1} disjoint, two-dimensional subspaces. Specifically, each subspace groups pairs of amplitudes whose binary representations differ only at the target qubits position q . For each pair the first element is at base index n_i where qubit q has a value of zero and the second element at index $n_i + 2^q$.

Each pair can then be multiplied independently by the base 2×2 gate matrix itself, before writing the resulting vectors elements back into their original positions in the statevector [7]:

$$\begin{pmatrix} a_{n_i} \\ a_{n_i+2^q} \end{pmatrix} \mapsto U \begin{pmatrix} a_{n_i} \\ a_{n_i+2^q} \end{pmatrix}$$

Iterating through the 2^{n-1} pairs means each amplitude in the statevector is operated on once. Applying the gate to each pair is mathematically equivalent to applying the full-system matrix to the entire statevector in one operation.

The resulting amplitudes can be written back into the statevector while the two necessary original amplitudes and the four matrix coefficients are already available in memory. This removes any non-constant space complexity terms that are being added to the statevectors significant $O(2^n)$ space complexity during gate application.

However, while DPM improves on the full-system strategy's cost, is there a strategy that can decrease memory further?

2.7 Gottesman-Knill theorem

The GK theorem poses the question: can the state of a quantum system be represented more compactly and if so for what classes of circuits does this hold?

The answer is yes, with stabilizer circuits. This is a class of circuit that is composed entirely of Clifford gates, a set of gates that includes but is not limited to: the Pauli X , Y and Z gates, the Hadamard gate and the CNOT gate. The set of possible states allowed by such gates allows a different representation, rather than tracking the amplitudes of every basis state, it can be described by the Pauli operators that stabilise it.

For a state $|\psi\rangle$ an operator S is a stabiliser if:

$$S|\psi\rangle = |\psi\rangle$$

so, if it leaves the state unchanged.

Note: As wierdly worded as GK theorem is, find way to reword this, add source. Expand on this if there is time, if not remove.

3 Design & Implementation

If I had to try and put my finger on my biggest issues with this, it's that I don't seem to be able to find a structure that makes sense or feels natural. I'm also struggling to put clear boundaries between higher and lower level aspects.

3.1 Development environment

Rust was chosen as the primary development language for its performance and safety guarantees while still allowing unsafe, direct memory management if necessary. Its zero-cost abstractions allow higher level intuitive interfaces to be used without causing additional runtime overhead.

Git was used for version control throughout so that branches could be used to develop experimental features in isolation from the main implementation. Additionally this allowed individual experiments and benchmarks to be associated with specific commits for reproducibility.

GitHub was used to host the source code and provide continuous integration through GitHub Actions. Automated unit tests were run on pushes to the default branch, on Windows and Ubuntu environments, providing correctness checks across platforms, ensuring that changes did not introduce subtle logic bugs and or unnacceptably compromise accuracy.

3.2 Library Architecture & Implementation

A software library was selected as the project format. This way the simulation can be exposed through a programmatic interface rather than being directly coupled to a particular application or user interface. In future it could be extended with a simulator application that utilises the library while the functionality remains independently accessible.

The library can be organised into three principle layers: the circuit representation, simulation state objects and kernels. The circuit representation describes the operations to be executed, simulation objects maintain simulation state and coordinate execution and kernels implement the mathematical transformations required by operations.

3.2.1 Circuit Representation

This project separates the description of a quantum circuit from the statevector object used to simulate its execution. Quantum operations are represented as instructions defining the operations themselves and contain parameters pertaining to non-implementation specific information. For example rotation gate angles and qubit targeting indices. This results in a clean interface that allows introducing alternative simulation objects with different state representations in the future, such as stabilizer-based simulation without requiring changes to the circuit representation.

The core component of this interface is the Instruction enum which provides variants describing different quantum operations, mainly gates. Each variant defines the parameters that may accompany it explicitly, creating a fixed structure for required inputs. The use of an enum also offers a common type the operations can be stored and processed through, making it straightforward to express an entire circuit as an ordered collection of instructions. That ability naturally lends itself to future extensions such as circuit level optimisation as the sequence could be analysed and transformed before execution.

3.2.2 Statevector

A simulation starts by initialising a simulation object such as a statevector, with the desired number of qubits. That statevector then owns the quantum state being simulated using a heap-backed buffer of amplitudes and supplies the interface through which operations can be applied. To progress the simulation an instruction or a circuit as a collection of instructions must be passed to the statevector for execution.

The statevector then acts as a validation and dispatch layer above the operations themselves during execution, while owning the amplitude buffer. By first matching on the instruction the statevector can extract the parameters to perform the necessary validation checks. Validation within the instructions themselves was considered, but checks such as target qubit bounds depend on the state itself. Keeping it in the statevector avoids splitting related checks between the instructions and the statevector object.

After validation, the statevector translates the qubit indices into statevector strides for DPM, its primary simulation strategy, where a stride represents the distance between corresponding amplitudes in the amplitude buffer. This is determined by the position of the target qubit within the chosen big-endian structure. Despite appearing to be a low-level implementation detail, generating the strides in this layer, decouples the kernels that will receive them from the memory layout of the buffer.

The statevector also encapsulates a performance configuration that specifies the hardware features to be utilised during execution. This configuration can be resolved automatically at runtime by detecting the host's CPU capabilities or it can be supplied explicitly to request a specific configuration, which is then validated against available host features. This validation is necessary because executing a kernel that uses unsupported hardware features can result in undefined behaviour.

Qubit measurement is handled directly by the statevector and follows a different execution path from instructions. Unlike unitary operations such as gate application, it returns a value and the circuit representation has no mechanism for collecting results, so measurement does not fit into it naturally. A measurement first calculates the marginal probability of the target qubits outcomes and then samples the outcome by comparing a random number against the probability of it being zero. It then

collapses the state by setting amplitudes representative of the false outcome to zero renormalising the remaining amplitudes. Both the probability calculation and the collapse step traverse the amplitude buffer by target stride in the same manner as the DPM kernels. They are currently both implemented directly in the statevector with no kernel dispatch but could be separated into kernels in a future implementation as both could benefit steps could benefit from hardware acceleration.

3.2.3 Kernels

The kernels are separated from the main statevector implementation in a submodule containing four implementations: portable, FMA and AVX. Each performs equivalent operations using different levels of hardware-specific optimisation. The portable implementation provides the DPM kernels and, for reference, the full-system matrix kernels, the FMA implementation provides DPM kernels that use fused multiply-add operations for arithmetic and the AVX implementation provides SIMD single-qubit kernels that adapt DPM but to process multiple amplitude pairs simultaneously. In order to simplify dispatch, all kernels share the same function signatures: a mutable slice of the amplitude buffer, a four element matrix and a target stride calculated by the statevector (controlled two-qubit kernels also require a control stride).

3.2.3.1 Portable DPM kernels

This is the baseline DPM implementation designed to be the baseline strategy executed with no hardware feature requirements. It can conceptually be split into two categories, functions that traverse the amplitudes and identify pairs of amplitudes and a function that, given these pairs and a matrix, performs the matrix-vector multiplication.

The single-qubit gate kernel consists of an outer loop that walks through the statevector in blocks. Each block contains two halves, each the length of a target stride, corresponding to a different value of the target qubit. An inner loop then traverses the first half of each block pairing each amplitude with the corresponding amplitude one target stride ahead.

This is especially giving me great difficulty to explain, maybe I can come up with a diagram:

The controlled two-qubit gate kernel adds one further condition: only amplitudes pairs in which the value of the control qubit is one can be updated. The traversal depends on which stride is larger. If the control stride is larger, the first loop steps through the amplitudes in blocks the size of two control strides. Then within the second half of the control block where the control qubit is one, the single-qubit gate traversal can be applied unchanged. If the target stride is larger then the outer loop traverses the amplitudes in blocks the length of two target strides, before a second loop walks through each half where the target qubit is zero and a final inner loop selects amplitudes in sub-blocks where the control is one and pairs each amplitude with another one a target stride ahead.

Matrix application for the kernels is delegated to a dedicated helper function that is called once per amplitude pair. It copies the pairs by value into a buffer on the stack, initialises a second stack buffer to zero for the results, performs the matrix-vector multiplication and then overwrites the original amplitudes with the results. This function can be considered the most performance-critical point in the program as it runs once for every updated pair, so keeping the pair and its results on the stack avoids any execution time penalties due to heap allocations.

3.2.3.2 FMA DPM kernels

A set of kernels with traversals algorithmically identical to the portable ones but with arithmetic changes to the matrix application. During matrix-vector application fused multiply-add functions are used. These functions only guarantee the result and not the use of hardware FMA instructions. Therefore, the matrix application function is therefore explicitly compiled with hardware FMA instructions using compiler directives. The DPM traversals must be duplicated in this implementation in order to compile them with the same directives since despite not containing FMA instructions it is required for inlining.

3.2.3.3 AVX DPM kernels

This is especially giving me great difficulty to explain, maybe I can come up with a diagram:

The AVX kernel follows the same traversal as the portable DPM kernel but processes two amplitude pairs per iteration using 256-bit SIMD vectors. Each vector holds four 16-byte double precision floating point values which is enough for two complex amplitudes stored as real and imaginary components. It is compiled with the AVX compiler directive to ensure that the processors SIMD YMM registers are used rather than the vectors being split across two 128-bit registers.

Updating two neighbouring low amplitudes together requires that both are in the first half of the same block, as in order for them to be loaded simultaneously they must be contiguous in memory, so the target stride must be at least two. A target stride of one means that every pair is interleaved in memory, so must be handled by other kernels instead. The outer loop walks the statevector in blocks of two target strides while the inner loop steps through the first half of each block in steps of two. Each iteration therefore updates two consecutive low amplitudes along with their respective partners one target stride ahead.

The matrix is prepared once before the traversal begins, each of the eight values that make up the four complex matrix coefficients are copied across every lane of their own vectors. This is so that a single vector multiplication can apply the same coefficient to both amplitudes held in a register. As the matrix values are invariant across the entire gate application these vectors are created once per kernel call and passed to the pair routine rather than rebuilding them for every pair. An additional

vector containing alternating signs used during complex multiplication is prepared in advance as well.

The main role of the kernel is to multiply a vector of amplitudes by one complex matrix coefficient. This is done by multiplying the amplitude vector by the real coefficient vector, and by the imaginary coefficient vector, both products are then stored separately in their own vectors. To facilitate complex multiplication the lanes of the imaginary products are swapped with their neighbouring lane, before adding the two product vectors and multiplying the result by the sign vector.

The matrix-vector product can then be constructed directly from this operation. The two low amplitudes are packed into a vector and the two high amplitudes into another. The updated low amplitudes are the sum of the top left coefficient applied to the low vector and the top right coefficients applied to the high vector. The updated high amplitudes follow the same process but with the bottom left and right coefficients respectively. Then the vectors are unpacked and written back to the amplitude buffer.

3.3 Testing, benchmarking and observability

Unit tests were used constantly throughout every step of development to ensure the behaviour of the simulator was as expected and inaccuracies were not introduced. Due to the use of floating point arithmetic a certain degree of accumulated numerical error is inevitable. However it was required to remain within a tolerance range of 10^{-14} throughout tests. The Rust ecosystem also provided tools for investigating performance.

Criterion.rs, a Rust port of the popular Haskell Criterion package, was used for execution time benchmarking and analysis. It iterates over benchmarks gathering samples and then provides a full statistical breakdown. It also supports direct comparison benchmarks useful for verifying the difference between different approaches and algorithms. This was most useful when it came to developing different gate application algorithms and analysing the effects of compiler directive placement.

DHAT was used to investigate the behaviour of heap allocations. It is registered as the global allocator and wraps the default allocator so that every allocation and deallocation is recorded while profiling is active. From this it can report the total number of allocations and bytes requested alongside peak heap usage. Each allocation also records the backtrace of the call that made it, which aids when analysing what these allocations can be attributed to.

The Tracing crate was used to provide observability insights into circuit execution. Primarily useful when examining sub-routines such as the quantum Fourier transform which expands into a series of gates whose ordering depends on parameters such as qubit count. It provided quality-of-life features such as exporting trace data to Chrome Perfetto for examination.

Rust's feature gating was used extensively throughout in coordination with these features to ensure that any accommodations for benchmarking and tracing did not introduce unnecessary overhead when unactivated.

4 Results

This chapter compares the correctness and computational cost of the various algorithms and approaches taken. It examines how execution time and memory usage evolve over an increasing number of qubits.

4.1 Experimental configuration & Methodology

The following results were achieved using a desktop computer equipped with a Ryzen 5 1600 processor and 16 GB of RAM. The operating system was CachyOS, a Linux distribution. The benchmarks were compiled and run on the nightly-2026-05-27 Rust toolchain.

Note: Provide a commit hash on main branch with all experiments ready to run

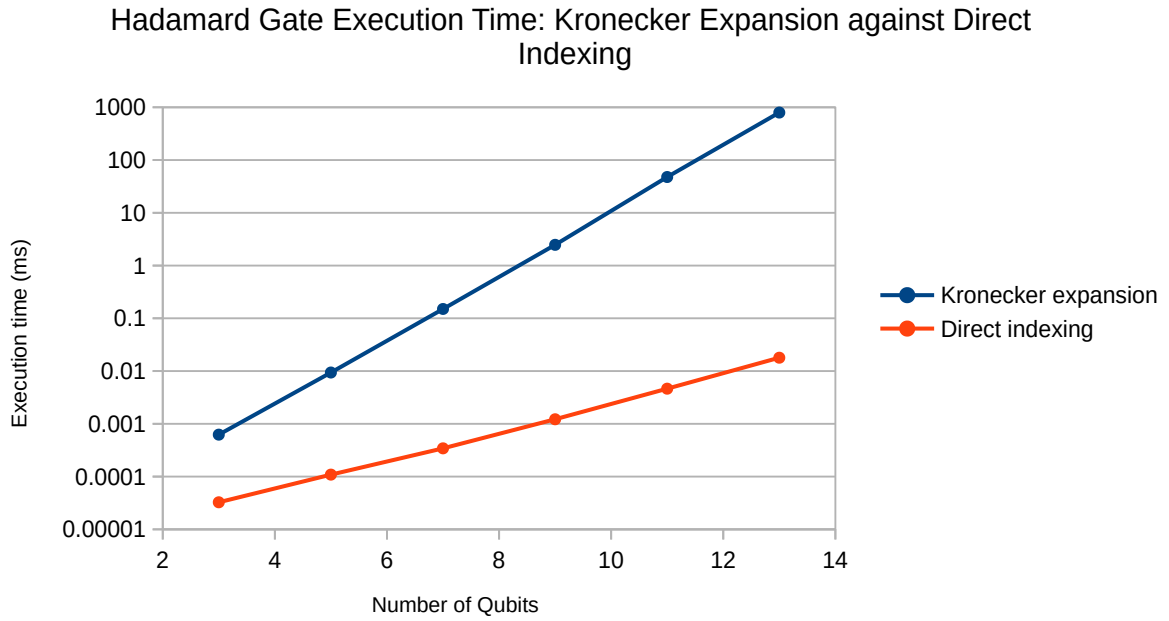
Each experiment that measured execution time was run using Criterion's benchmarking harness. Before collecting measurements, Criterion performed a three second warm-up period during which the benchmark is repeatedly executed without measurement in order to allow the system and host machines hardware to adapt to the load. It then ran benchmarks repeatedly over thousands of iterations to obtain 100 samples. It then provided a statistical analysis of said samples providing a mean, standard deviation and confidence intervals.

Experiments that measured memory usage were run using DHAT which provides heap profiling.

4.2 Full-system matrix against direct indexing

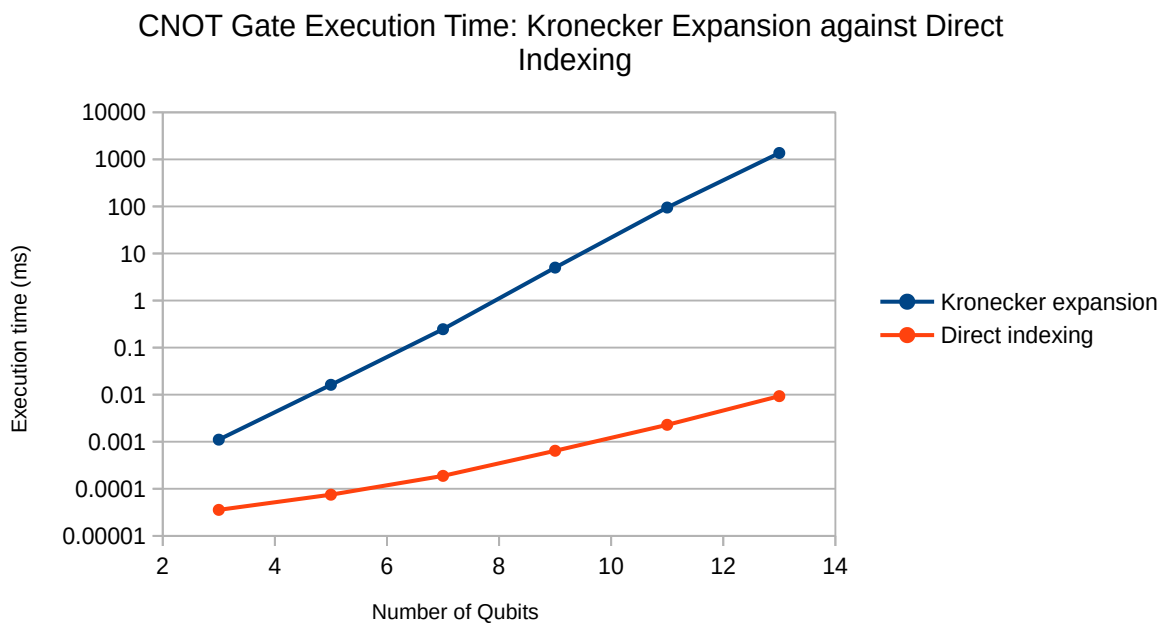
The first comparison evaluates expansion of the full-system matrix against the direct indexing approach. The two approaches were benchmarked using Hadamard and CNOT gates over an increasing number of qubits. This covers both the single-qubit and controlled two-qubit kernels. To isolate qubit scaling as the independent variable, the middle qubit was always selected as a target in both kernel runs. In the CNOT benchmarks the control qubit was always selected as zero.

4.2.1 Hadamard gate execution time



The difference between the two implementations increases exponentially with the number of qubits. Initially at $n = 3$ the direct indexing exhibits a speedup of magnitude 19.13, by $n = 13$ this factor is 44,391.94. The performance gap widens significantly as qubit count increases as the full-system matrix expansion becomes more expensive.

4.2.2 CNOT gate execution time

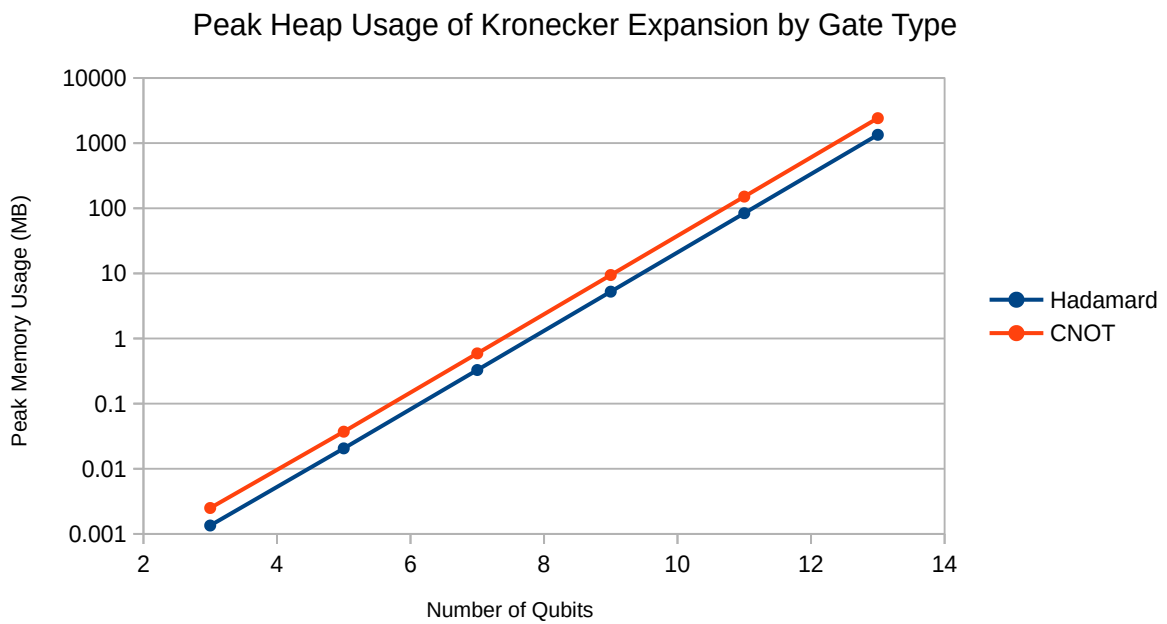


The controlled two-qubit variants of the kernels show an even larger difference. At $n = 3$ a speedup of 31.03 is observed and at $n = 13$ this has reached a factor of 147,010.09.

Note: Lots to talk about here should be the same as Hadamard with a difference of 4

4.2.3 Hadamard and CNOT gate peak memory usage

Direct indexing was excluded from the following graphs, since as expected, it incurs no additional allocations aside from the statevector itself. Instead the difference in memory usage between the two full-system matrix expansion kernels can be examined.



At $n = 3$ the controlled two-qubit kernel consumes 1.86 times the peak memory as the single-qubit kernel. At $n = 13$ this factor had approached 1.80. At this point the single-qubit kernel reaches a peak memory usage of approximately 1.34 GB, while the controlled two-qubit reaches 2.42 GB.

4.3 Hardware accelerated kernels

The following experiments evaluate the effect of fused multiply-add operations and SIMD instructions on the execution time of direct-indexed statevector gate application.

4.3.1 Hadamard gate execution time over number of qubits

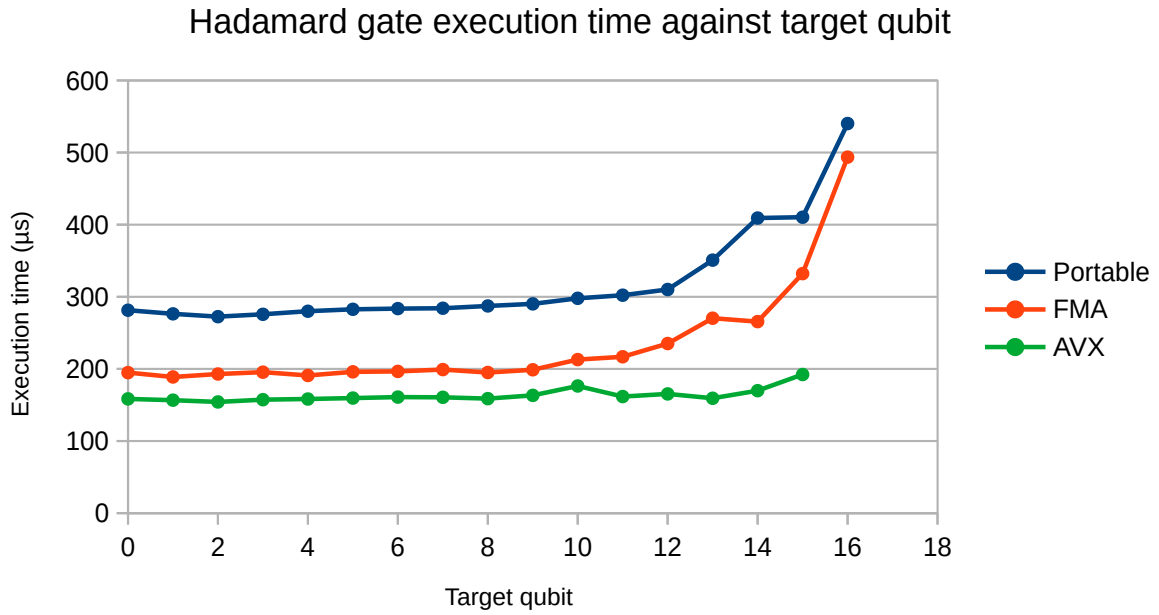
Applying a Hadamard gate over an increasing number of qubits with the middle qubit always selected as a target to isolate qubit scaling gave the following data:



All three implementations show the expected exponential increase in execution time as qubit count increases. However both hardware accelerated kernels consistently outperform the portable kernel, with a significant reduction in execution time from the AVX kernel. At $n = 3$ the speedup from the FMA and AVX kernels are 1.12 and 1.80 respectively while at $n = 19$ this has increased to 1.40 and 1.58.

4.3.2 Hadamard gate execution time over target qubit

This experiment differs in the last in that the circuit size remained constant at $n = 17$, this time the independent variable was the target qubit. Each kernel was aside from AVX was tested at all target qubit positions from zero to $n - 1$. The AVX kernel cannot accept a target qubit equal to $n - 1$ since the target amplitudes are interleaved with a stride equal to one. Running this experiment provided the following results:



The AVX implementation once again, consistently outperforms the other implementations, at every measurement value it can be applied to. There is an evident increase in execution time as the target approaches $n - 1$. However on the final three targets there are peaks at $n - 3$ and $n - 1$ with a trough at $n - 2$. This pattern was reproduced at various constant qubit counts.

Note: Run at different odd/even circuit size

5 Analysis and discussion

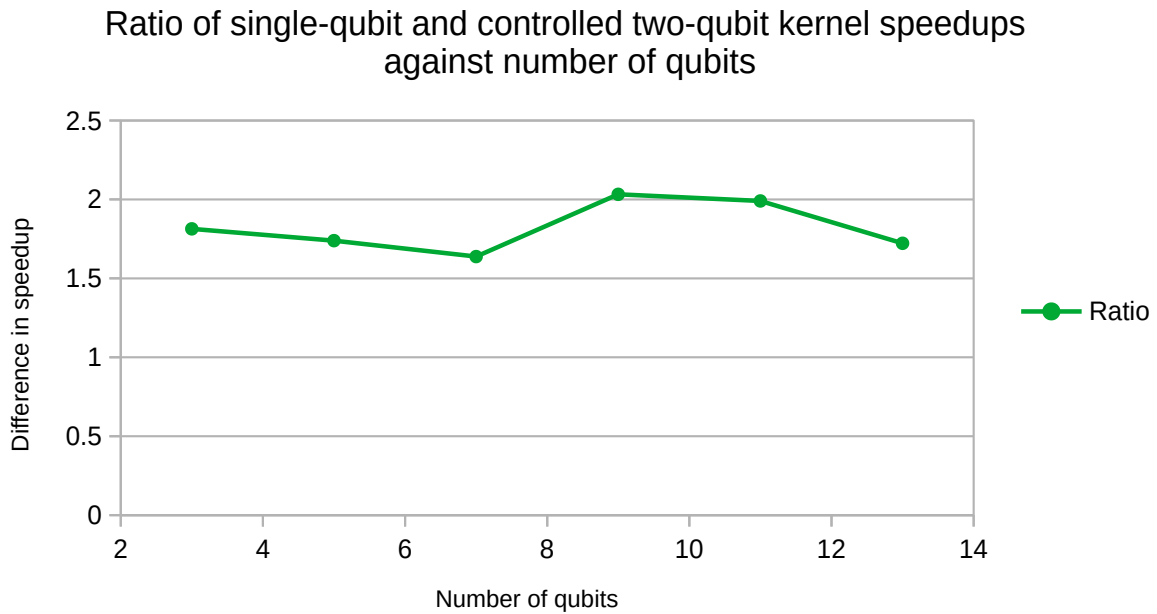
5.1 Full-system matrix expansion

The execution time results show that for both types of kernel the full-system matrix expansions cost becomes increasingly impractical. Primarily the memory usage is what limits this approach.

In the peak memory usage benchmarks, as direct indexing did not trigger any additional allocations the total memory usage remained at the size of the statevector, approximately 131 KB at $n = 13$. In comparison, the full-system matrix implementations require far more memory to construct and apply the expanded matrix. At $n = 13$, this reached approximately 1.34 GB for the single-qubit kernel and 2.42 GB for the controlled two-qubit kernel. This demonstrates that full-system matrix expansion becomes impractical quickly at relatively small circuit sizes due to its rapidly increasing memory requirements.

The difference in execution time also follows the same underlying scaling issue. A full-system matrix contains $2^n \times 2^n = 4^n$ elements, applying these to the statevector through matrix-vector multiplication takes time as well as memory.

The “CNOT” results show the same behaviour as the Hadamard results, however the difference between the two kernels is even larger. This is due to the opposite approaches they take to conditionals. Full-system matrix expansion requires two system matrices multiplying the cost by a factor of two, while the direct indexing kernel actually takes advantage of the control aspect to reduce the amount of work. Since the direct approach individually selects qubit pairs at a time to operate on, it can calculate the positions of the $c = 1$ amplitudes and construct a loop condition that skips all $c = 0$ amplitudes, in theory halving the workload. Due to this we should expect to see the difference in execution time between the two kernels multiply by four when comparing the Hadamard and CNOT experiments.



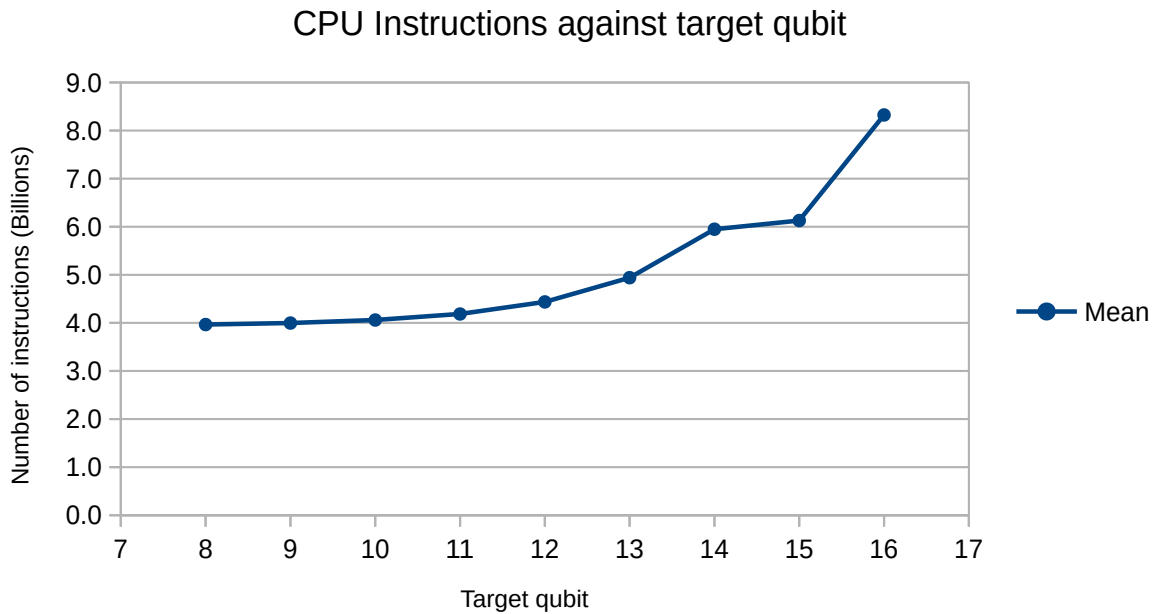
Interestingly that prediction does not hold with the difference in speedup hovering around a modest factor of two.

5.2 Hardware acceleration

The hardware-accelerated kernels consistently reduced execution time across both experiments compared with the portable implementation. FMA showed a modest improvement, while the AVX implementation provided the largest reduction in execution time.

The target experiment revealed an established pattern as the target qubit increased. Execution time surges as the x-axis approaches the final three to four qubits, despite no change in the amount of data processed. The only variable that has changed is how the accesses are distributed in memory.

After further analysis using `perf`, a tool for recording hardware counters such as instruction counts and cache activity the following was found:



This data was obtained by recording instruction count over the execution of a binary that applied one thousand Hadamard gates to a 17 qubit statevector. Five measurements were taken for each target qubit in order to reduce variability introduced by runtime overhead. However maximum the standard deviation is only 0.0000231% at target qubit 9.

Note: Explain target feature vs algorithmic speedup - 30m

6 Conclusion

Note: Should only take 1-2h at most, write on tuesday morning once experiment selection confirmed, 2-3 paragraphs.

- Confirmed speedup and lighter memory requirements of direct pair approach.
- Confirmed limited speedup from hardware acceleration.

Bibliography

- [1] S. Aaronson, “THE LIMITS OF Quantum,” *Scientific American*, vol. 298, no. 3, p. 62, 2008, Accessed: Apr. 20, 2026. [Online]. Available: <http://www.jstor.org/stable/26000518>
- [2] J. W. Z. Lau, K. H. Lim, H. Shrotriya, and L. C. Kwek, “NISQ computing: where are we and where do we go?,” *AAPPS Bulletin*, vol. 32, no. 1, Sep. 2022, doi: <https://doi.org/10.1007/s43673-022-00058-z>.
- [3] G. S. Ravi, K. N. Smith, P. Gokhale, and F. T. Chong, “Quantum computing in the cloud: Analyzing job and machine characteristics,” in *2021 IEEE International Symposium on Workload Characterization (IISWC)*, IEEE, 2021, pp. 39–50.
- [4] A. Cicero, M. A. Maleki, M. W. Azhar, A. F. Kockum, and P. Trancoso, “Simulation of quantum computers: Review and acceleration opportunities,” *ACM Transactions on Quantum Computing*, vol. 7, no. 1, pp. 1–35, 2025.
- [5] O. R. L. C. Facility, “Summit.” [Online]. Available: <https://www.olcf.ornl.gov/summit/>
- [6] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*, 10th anniversary edition. Cambridge: Cambridge university press, 2010.
- [7] T. Jones, A. Brown, I. Bush, and S. C. Benjamin, “QuEST and high performance simulation of quantum computers,” *Scientific reports*, vol. 9, no. 1, p. 10736, 2019.

Appendix

Qubits	Full-system matrix (ms)	Direct indexing (ms)
3	0.000623	0.000033
5	0.009367	0.000109
7	0.15016	0.000343
9	2.469403	0.001218
11	47.624858	0.004629
13	793.488575	0.017875

Table 1: Hadamard gate execution time for full-system matrix expansion and direct indexing.

Qubits	Full-system matrix (ms)	Direct indexing (ms)
3	0.001106	0.000036
5	0.016174	0.000075
7	0.245583	0.000188
9	5.016699	0.000642
11	94.779459	0.002286
13	1366.488455	0.009295

Table 2: CNOT gate execution time for full-system matrix expansion and direct indexing.

Qubits	Hadamard (MB)	CNOT (MB)
3	0.001344	0.002496
5	0.020544	0.037056
7	0.327744	0.590016
9	5.242944	9.437376
11	83.886144	150.995136
13	1342.177344	2415.919296

Table 3: Peak memory usage of full-system matrix expansion for Hadamard and CNOT gates.

Qubits	Portable (μ s)	FMA (μ s)	AVX (μ s)
3	0.032974	0.029392	0.018314
5	0.107920	0.073119	0.047978
7	0.352090	0.271980	0.164130
9	1.205300	0.918370	0.622740
11	4.576000	3.254700	2.513300
13	18.508000	12.715000	10.356000
15	72.479000	49.887000	41.201000
17	280.930000	202.460000	163.710000
19	1203.100000	857.450000	760.220000

Table 4: Hadamard gate execution time for hardware-accelerated kernels.